Part 1: Teams as the Means of Delivery

Chapter 1: The Problem with Org Charts

"The organizational structure must coordinate accountabilities to support the goals of delivering high-quality, impactful software"

Systems thinking focuses on optimizing for the whole, looking at the overall flow of work, identifying what the largest bottleneck is today, and eliminating it. Then repeat.

Three Organizational Structures in Every Organization:

- Formal Structure (org chart) - facilitates compliance
- Informal Structure - the "real of infuence" between individuals
- Value Creation Structure - how work actually gets done based on inter-personal and inter-team reputations

Five Rules of Thumb for Designing Organizations:

- Design when there is a compelling reason
- Develop options for deciding on a design
- Choose the right time to design
- Look for clues that things are out of alignment
- Stay alert to the future

Four Fundamental Team Types:

- stream-aligned
- platform

- enabling
- complicated-subsystem

Three Core Team Interaction Modes:

- Collaboration
- X-as-a-Service
- Facilitating

Conways Law: "Organizations which design systems... are constrained to produce designs which are copies of the communication structures of these organizations"

ie: There is a strong correlation between an Organizations real communications paths (value creation structure) and the resulting software architecture.

"If you have 4 groups working on a compiler, you'll get a 4-pass compiler"

Three Elements of Intrinsic Motivation:

- Autonomy
- Mastery
- Purpose

We need to put the team first, advocating for restricting their cognative load.

Chapter 2: Conway's Law and Why it Matters

"If the architecture of the system and the architecture of the organization are at odds, and the architecture of the organization wins"

An organization that is arranged in functional silos is unlikely to ever produce software systems that are well architected for end to end flow.

Reverse Conway Maneuver: *Organizations should evolve their team and organizational structure to achieve the desired architecture. The goal is for your architecture to support the ability of teams to get their work done - from design through to deplovement - without requiring high bandwidth communication between teams.*

Ex:

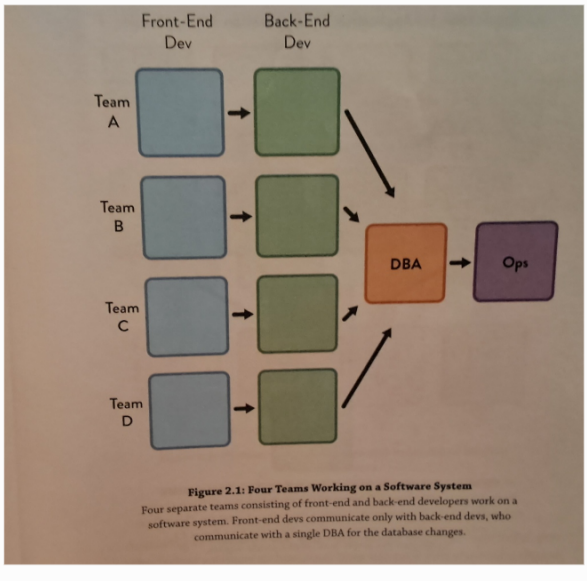

Figure 2.1: Four Teams Working on a Software System
Four separate teams consisting of front-end and back-end developers work on a software system. Front-end devs communicate only with back-end devs, who communicate with a single DBA for the database changes.

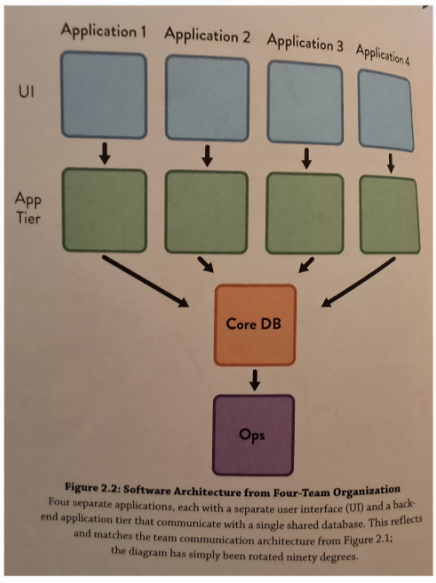

Figure 2.2: Software Architecture from Four-Team Organization
Four separate applications, each with a separate user interface (UI) and a back-end application tier that communicate with a single shared database. This reflects and matches the team communication architecture from Figure 2.1; the diagram has simply been rotated ninety degrees.

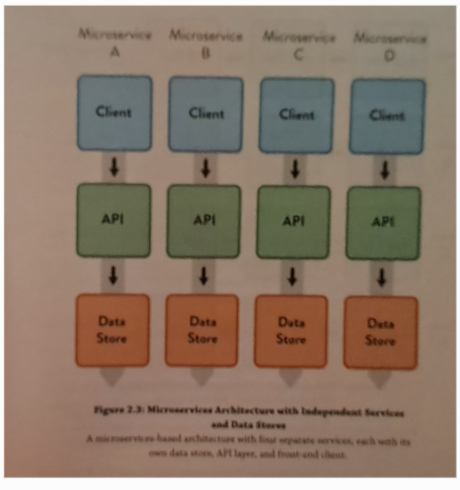

Figure 2.3: Microservices Architecture with Independent Services and Data Stores
A microservices-based architecture with four separate services, each with its own data store, API layer, and front-end client.

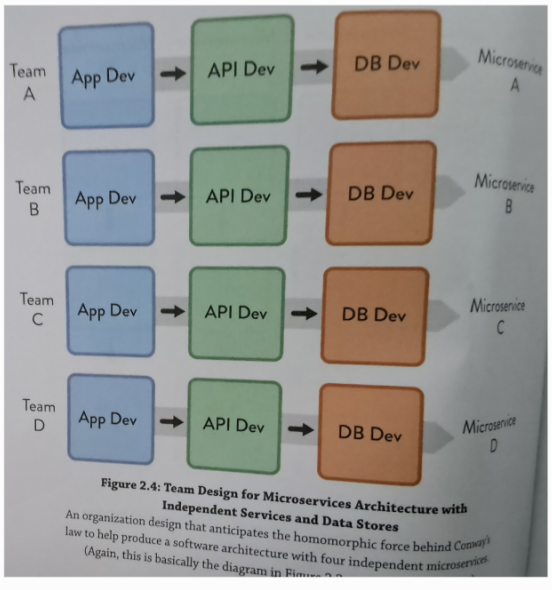

Figure 2.4: Team Design for Microservices Architecture with Independent Services and Data Stores
An organization design that anticipates the homomorphic force behind Conway's law to help produce a software architecture with four independent microservices. (Again, this is basically the diagram in Figure 2.2.)

"Team assignments are the first draft of the architecture"

Software architecture good practices:

- Loose coupling - components do not hold strong dependencies on other components
- High cohesion - components have clear bounded responsibilities, and their internal elements are strongly related
- Clear and appropriate version compatibility
- Clear and appropriate cross-team testing

"We can also exploit architecture as an enabler of rapid change. We do this by partitioning our architecture to gracefully absorb change."

"If we have managers deciding... which services will be built, by which teams, we implicitly have managers deciding on the system architecture"

More comunnication between teams is not always better. Focused communication is what is needed. Look for unexpected communications and address the cause.
(Kevan: I get the theory behind this, but it makes me feel... icky)

"Managers should focus their efforts on understanding the causes of unaddressed design interfaces... and unpredicted team interactions... across modular systems"

(Kevan: I think this is really getting at how to avoid spagetti code. If everyone need to communicate with everyone else, when your architecture will become a complicated tangled mess.)

Chapter 3: - Team First Thinking

"Disbanding high-performing teams is worse than vandalism: it is corporate psychopathy."

Teams working as a cohesive unit perform far better than collections of individuals for knowledge-rich, problem-solving tasks that require high amounts of information

Google found for their own teams, who is on a team matters less than the team dynamics; and that when it comes to measuring performance, teams matter more than individuals.

Team: a stable group of 5 to 9 people who work toward a shared goal as a unit.

High trut is explicitly valued and designed for. High trust is when enables a team to innovate and experiment.

- Team 5-9 (No more than 15)
- Families ("Tribes") no more than 50 people
- Divisions/Streams/Profit & Loss lines groupings of no more than 150 or 500 people

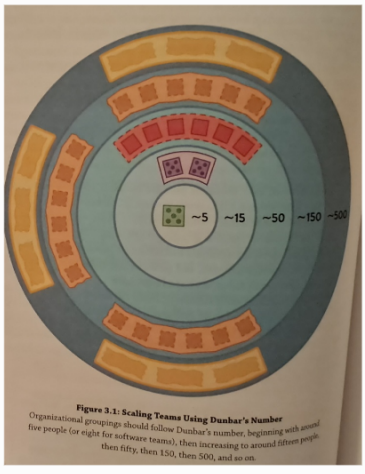

Figure 3.1: Scaling Teams Using Dunbar's Number
Organizational groupings should follow Dunbar's number, beginning with around five people (or eight for software teams), then increasing to around fifteen people, then fifty, then 150, then 500, and so on.

In high-trust organizations people can switch teams every 9-12 months. In organizations with lower levels of trust people should remain on the same team for 18 months-2 years

Tuckman Teal Performance Model desribes how teams perform:

1. Forming: Assembling for the first time
2. Storming: Working through initial differences in personality and ways of working
3. Norming: Evolving standard ways of working together
4. Performing: Reaching a state of high effectiveness

Every part of the software system needs to be owned by exactly one team. This mean there should be no shared ownership of components, libraries, or code. Teams may use shared services at run time, but every running service, application, or subsystem is owned by only one team. Outside teams may submit pull requests, or suggestions for change to the owning team, but they cannot make the change themselves.

(Kevan: I'm pretty sure our team really really doesn't do this at the moment.)

Ownership of code should not be a territorial thing. The team takes responsibiliy for the code, and cares for it, but individual teams members should not feel like the code is theirs to the exclusion of others. The team should view

themselves as stewards or caretakers as opposed to private owners. Think gardeners not police.

Team members should put the needs of the team above their own. They should:

- Arrive for stand ups and meetings on time
- Keep discussions and investigations on track
- Encourage a focus on team goals
- Help unblock other team members before starting new work
- Mentor new or less experienced team members
- Avoid "winning" arguements and instead agree to explore options

Some people are unwilling to put tesm needs above their own or are unsuited to work on teams. These people are team toxic, and need to be removed before damage is done.

A software subsystem should not be allowed to grow beyond the cognative load of the team responsible for the software.

Cognative Load: "The total amount of mental effort being used in the working memory"

Three different kinds of cognative load:

1. Intrinsic Cognative Load - related to aspects of the task fundamental to the problem space (ie: "What is the structure of a Java class" "How do I create a new method")
2. Extraneous Cognitive Load - relates to the environment in

which the task is being don. (ie "How do I deploy this component again?" "How do I configure this service")
3. Germane Cognative Load - relates to aspects of the task that need special attention for learning or high performance (ie "How should this service interace with the ABC service")

One way to measure cognative load, ask the team "Do you feel like you're effective and able to respond in a timely fashion to the work you are asked to do"
(Kevan: Maybe this should be a part of our teams retros?)

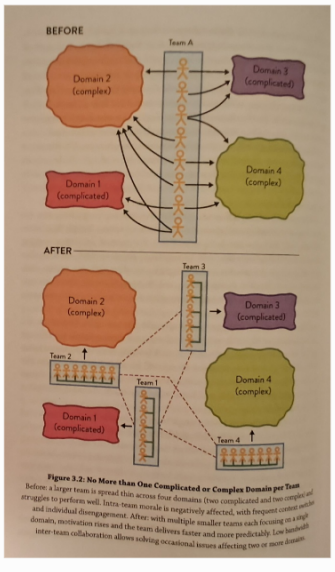

Figure 3.2: No More than One Complicated or Complex Domain per Team
Before: a larger team is spread thin across four domains (two complicated and two complex) and struggles to perform well. Intra-team morale is negatively affected, with frequent context switches and individual disengagement. After: with multiple smaller teams each focusing on a single domain, motivation rises and the team delivers faster and more predictably. Low bandwidth inter-team collaboration allows solving occasional issues affecting two or more domains.

Instead of designing a system in the abstract, we need to design the system and it's software boundaries to fit the available cognative load within delivery teams.

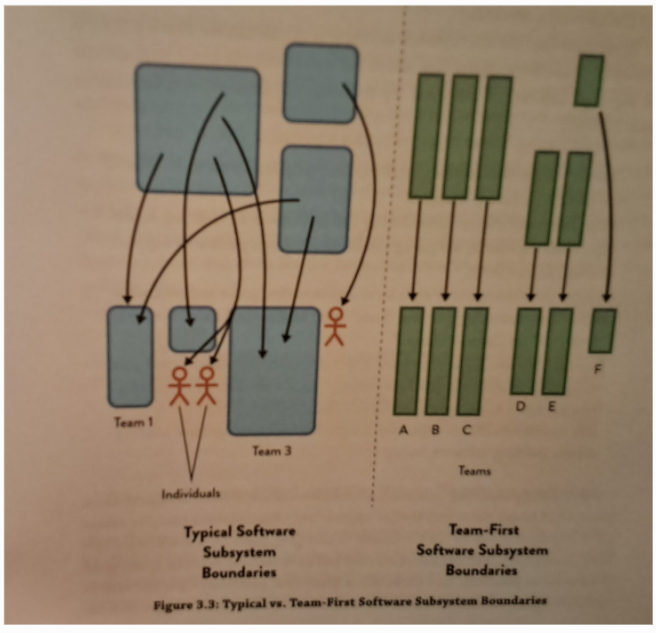

Figure 3.3: Typical vs. Team-First Software Subsystem Boundaries

Team APIs:

- Code: Runtime endpoints, libraries, clients, UI, etc. produced by the team
- Versioning: How the team communicates changes to it's code and services (ie: using semantic versioning as a "team promise" not to break things)
- Wiki and documentation: Especially how-to guides for the software owned by the team
- Practices and principles: The team's preferred ways of working
- Communication: The teams approach to remote communication tools, such as chat or video conferencing.
- Work Information: What the team is working on now, what's coming next, and overall priorities in the short to medium term
- Other: Anything else that other teams need to use to interact with the team

The team API should expicitly consider usability by other

teams: Will other teams find it easy and straightforward to interact with us, or will it be difficult or confusing? How easy will it be for a new team to get on board with our code and working practices? How do we respond to pull requests and suggestions from other teams? Is our backlog and product roadmap easily visible and understandable by other teams? (Kevan: This is something we can definitly do better at. Wondering how much input does FW have in HH work)

Kevan: The book went through several case studies on office layout. Almost all stressed the need for squads (teams) to sit together. And tribes of squads working on the same topics near each other. Most stressed suggested squads be open concept within the squad but physically seperated from other squads. ie White board divider walls. One case study suggested having all squads in a tribe organized the same way, (ie kaban boards, etc) so different squads can see status at a glance. Most case studies suggested keeping squads small, breaking into smaller squads as work or teams grow. New squads inherit the original squads culture, limiting the "storming" and "norming" phases. Most case studies opted for large amounts of white boards, or drawing walls for collaboration, and lounges or event spaces. One case study had everything (even plants) on wheels to make it easy to move and reorganize. One limited technology, screens and personal items in work space. To make it easy to move around, and more difficult to hide.

Part 2: Team Topologies that Work for Flow

Chapter 4: Static Team Topologies

Two team building anti-patterns. Ad hoc teams and shuffling team members.

Organizations must design teams intentionally by asking these questions: Given our skill, constraints, culture and engineering maturity, desired software architecture and buisness goals which team topology will help us deliver results faster and safer? Where shoulde the boundaries be in the software system in order to perserve system viability and encourage rapid flow? How can our teams align to that?

"An organization has a better chance of success if it is reflectively designed"

Spotify model: 5-9 people make up a squads that work on a long term mission. Squads that work in similar areas for a tribe. Member of a tribe that share similar skills and practices (ie testers) share knowledge through a chapter. Guilds are used to share knowdledge across tribes

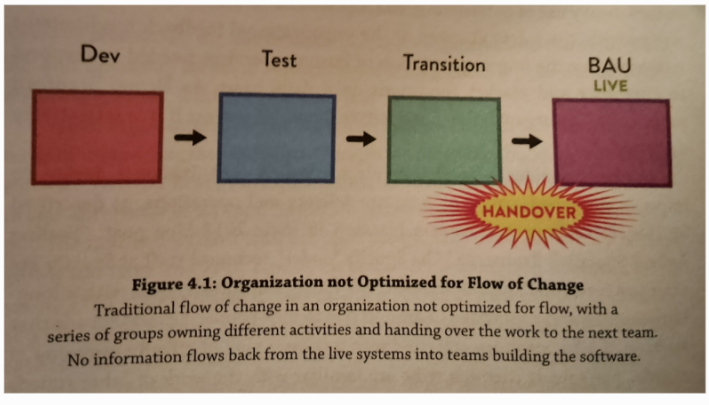

Figure 4.1: Organization not Optimized for Flow of Change
Traditional flow of change in an organization not optimized for flow, with a series of groups owning different activities and handing over the work to the next team. No information flows back from the live systems into teams building the software.

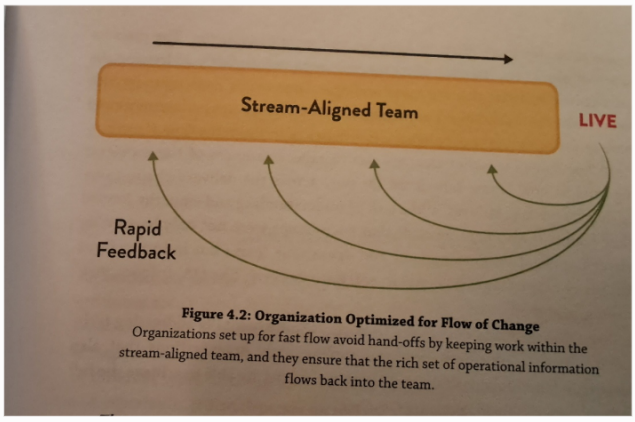

Figure 4.2: Organization Optimized for Flow of Change
Organizations set up for fast flow avoid hand-offs by keeping work within the stream-aligned team, and they ensure that the rich set of operational information flows back into the team.

The DevOps Topologies reflects two key ideas 1) There is no one-size-fits-all approach for structuring teams for DevOps success. The suitability and effectiveness of any given topology depends on the organization's context. 2) There are several topologies known to be detrimental (anti-patterns) to DevOps success, as they overlook or go against core tenents of Devops.

Ericsson example: Moved to devops approach in 2015, where each is resonsible for developing and supporting their software in production. Occasionally very large features are worked on simultaneously by a few colocated teams. Inter-team communication and dependencies were greatly reduced while working on intra-subsystem features. New roles are created to keep overight of the system to ensure subsystem integration, user experience, perfirmance, and reliability. System architects, system owners, and integration leads act as "communication conduits" with direct and requent interactions with the feature team.

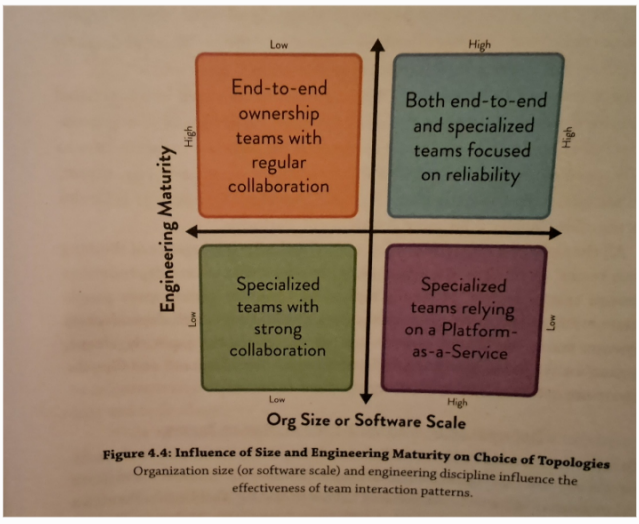

Figure 4.4: Influence of Size and Engineering Maturity on Choice of Topologies
Organization size (or software scale) and engineering discipline influence the effectiveness of team interaction patterns.

It is essential to detect and track dependencies and wait times between teams. It is recommended to use a Physical Dependency Matrix or "dependency tags" on kanban cards to identify and track dependencies, and infer the communication needed to make these dependencies work well. "**Visualizing important cross team information helps communicate across teams**"

Three different categories of dependency: knowledge, task, and resource dependencies.

Chapter 5: The Four Fundamental Team Topologies